



第三周作业

算法分析题

2-7

设 $P(x) = a_0 + a_1x + \cdots + a_dx^d$ 是一个 d 次多项式。假设已有一个算法能在 $O(i)$ 时间内计算一个 i 次多项式与一个一次多项式的乘积，以及一个算法能在 $O(i \log i)$ 时间内计算两个 i 次多项式的乘积。对于任意给定的 d 个整数 $n_1, n_2, \cdots, n_d$ ，用分治法设计一个有效算法，计算出满足 $P(n_1) = P(n_2) = \cdots = P(n_d) = 0$ 且最高次项系数为 1 的 d 次多项式 $P(x)$ ，并分析算法的效率。

解：

$P(x)$ 是一个最高次项系数为 1 的 d 次多项式且满足 $P(n_1) = P(n_2) = \cdots = P(n_d) = 0$ ，则 $P(x) = (x - n_1)(x - n_2) \cdots (x - n_d) = \prod_{i=1}^d (x - n_i) = \prod_{i=1}^{\lfloor \frac{d}{2} \rfloor} (x - n_i) \prod_{i=\lfloor \frac{d}{2} \rfloor + 1}^d (x - n_i) = P_1(x)P_2(x)$

则求解 $P(x)$ 时，可使用分治法将其转化为两个 $d/2$ 次多项式的乘积

设算法所需时间为 $T(d)$ 则递归式为：

$$T(d) = \begin{cases} \Theta(1), & d = 1 \\ 2T(d/2) + O(\frac{d}{2} \log \frac{d}{2}) = 2T(d/2) + \Theta(d \log d), & d > 1 \end{cases}$$

其中每层两个 $d/2$ 次多项式的相乘的开销为 $\Theta(d \log d)$

由主定理， $\log_b a = 1, f(n) = \Theta(n \log n) = \Theta(n^{\log_b a} * \log n)$ ，故 $T(n) = \Theta(n \log^2 n)$

2-15

设有 n 个运动员要进行网球循环赛。设计一个满足以下要求的比赛日程表：

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
- (2) 每个选手一天只能赛一次；
- (3) 当 n 是偶数时，循环赛进行 $n-1$ 天。当 n 是奇数时，循环赛进行 n 天。

解：采用分治法，设该算法为 `def tourna_table(n: int)`

- 当 n 为奇数时，增加一个虚拟选手 $n+1$ ，调用 `tourna_table(n+1)` 使之变为复数问题。返回日程表后，去掉最后一行日程，保留前 n 行，排上虚拟选手的比赛代表轮空，满足“当 n 是奇数时，循环赛进行 n 天”
- 当 n 为复数时，递归构造 $n/2$ 的日程表（调用 `tourna_table(n/2)`），然后再把它复制、扩展成 n 个选手的日程表（假设该复制算法为 `def copy(n: int)`）
 - $m=n/2$ 为偶数时，将日程表分为左上（`table[1][1]~table[m][m]`）、右上、左下、右下四个部分，`tourna_table(n/2)` 得到左上部分日程表后，递归外层调用 `copy(n)`，将左上部分整体复制进右下部分，再讲每个元素与 m 相加，复制进左下/右上部分，即可得到 n 位选手的日程表
 - $m=n/2$ 为奇数时，需处理 `tourna_table(n/2)` 返回的轮空比赛：将选手 $1 \sim m+1, 2 \sim m+2 \cdots m \sim n$ 一一配对，前 m 个选手若遇到轮空比赛，令其与其对应的选手比赛；剩余天数/选手的比赛再进一步填补、对齐即可。
 - 边界条件: $n=1$ 时，返回 `table[1][1]=1` 即可

算法设计题、

2-6

2-6 排列的字典序问题。

问题描述： n 个元素 $\{1, 2, \dots, n\}$ 有 $n!$ 个不同的排列。将这 $n!$ 个排列按字典序排列，并编号为 $0, 1, \dots, n!-1$ 。每个排列的编号为其字典序值。例如，当 $n=3$ 时，6 个不同排列的字典序值如下：

字典序值	0	1	2	3	4	5
排列	123	132	213	231	312	321

算法设计：给定 n 及 n 个元素 $\{1, 2, \dots, n\}$ 的一个排列，计算出这个排列的字典序值，以及按字典序排列的下一个排列。

数据输入：由文件 input.txt 提供输入数据。文件的第 1 行是元素个数 n 。接下来的 1 行

是 n 个元素 $\{1, 2, \dots, n\}$ 的一个排列。

结果输出：将计算出的排列的字典序值和按字典序排列的下一个排列输出到文件 output.txt。文件的第 1 行是字典序值，第 2 行是按字典序排列的下一个排列。

输入文件示例

input.txt

8

2 6 4 5 8 1 7 3

输出文件示例

output.txt

8227

2 6 4 5 8 3 1 7

```
import math

class Solution:
    def __init__(self):
        self.perm_dict = {}

    def permute(self, nums: list[int]) -> list[list[int]]:
        self.perm_dict = {}
        self.perm(nums, 0)

    def perm(self, nums, k):
        if k == len(nums): # 仅在所有位置排列好时输出一次结果
            rank = self.calc_rank(nums[:])
            self.perm_dict[rank] = nums[:]
            return

        for i in range(k, len(nums)):
            nums[i], nums[k] = nums[k], nums[i]
            self.perm(nums, k + 1) # 产生k后位置的全排列
            nums[i], nums[k] = nums[k], nums[i] # 恢复现场

    def calc_rank(self, nums):
        n = len(nums) - 1
        rank = 0
```

```

        for i in range(n + 1):
            c_i = 0
            for j in range(i + 1, n + 1):
                if nums[i] > nums[j]:
                    c_i += 1
            rank += c_i * math.factorial(n - i)
    return rank

s = Solution()
with open("input.txt", "r", encoding="utf-8") as fin, open(
    "output.txt", "w", encoding="utf-8"
) as fout:
    n = int(fin.readline().rstrip("\n"))
    p = list(map(int, fin.readline().split()))
    nums = [i for i in range(1, n + 1)]
    s.permute(nums)
    index = s.calc_rank(p)
    result = s.perm_dict[index + 1]
    fout.write(str(index) + "\n")
    fout.write(" ".join(map(str, result)))

```

2.9

2-9 双色 Hanoi 塔问题。

问题描述：设 A 、 B 、 C 是 3 个塔座。开始时，在塔座 A 上有一叠共 n 个圆盘，这些圆盘自下而上，由大到小地叠放在一起。各圆盘从小到大编号为 $1, 2, \dots, n$ ，奇数号圆盘着红色，偶数号圆盘着蓝色，如图 2-19 所示。现要求将塔座 A 上的这一叠圆盘移到塔座 B 上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：

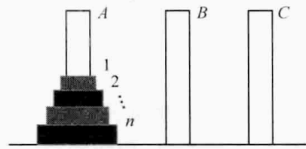


图 2-19 双色 Hanoi 塔

- 规则 I：每次只能移动 1 个圆盘；
- 规则 II：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；
- 规则 III：任何时刻都不允许将同色圆盘叠放在一起；
- 规则 IV：在满足移动规则 I ~ III 的前提下，可将圆盘移至 A 、 B 、 C 中任一塔座上。

试设计一个算法，用最少的移动次数将塔座 A 上的 n 个圆盘移到塔座 B 上，并仍按同样顺序叠置。

算法设计：对于给定的正整数 n ，计算最优移动方案。

数据输入：由文件 input.txt 给出输入数据。第 1 行是给定的正整数 n 。

结果输出：将计算出的最优移动方案输出到文件 output.txt。文件的每行由一个正整数 k 和 2 个字符 c_1 和 c_2 组成，表示将第 k 个圆盘从塔座 c_1 移到塔座 c_2 上。

输入文件示例
input.txt
3

输出文件示例
output.txt
1 A B
2 A C
1 B C
3 A B
1 C A
2 C B
1 A B

```
def t_name(n):  
    return chr(ord("A") + n)  
  
def dc_hanoi(n, src, dst, tmp, result): # result 用于缓存输出结果  
    if n == 1:  
        disk = towers[src].pop()  
        towers[dst].append(disk)  
        result.extend(" ".join((str(disk), t_name(src), t_name(dst), "\n")))  
    else:  
        dc_hanoi(n - 1, src, tmp, dst, result)  
        disk = towers[src].pop()  
        towers[dst].append(disk)  
        result.extend(" ".join((str(disk), t_name(src), t_name(dst), "\n")))  
        dc_hanoi(n - 1, tmp, dst, src, result)
```

```

with open("input.txt", "r", encoding="utf-8") as fin, open(
    "output.txt", "w", encoding="utf-8"
) as fout:
    n = int(fin.read())
    towers = [
        [(n - i + 1) for i in range(n + 1)],
        [],
        []
    ]
    result = []
    dc_hanoi(n, 0, 1, 2, result)
    for step in result:
        fout.write(step)

```

即原版hanoi递归算法 `hanoi(n,src,dst,tmp)`。

用数学归纳法证明其不违反规则III:

- 基础情形：
 - $n = 1$ 时，仅移动1号盘至空柱dst，不违反
 - $n = 2$ 时，移动步骤：

- `1 src→tmp`
- `2 src→dst`
- `1 tmp→dst`

同样不违反

- 假设对 $k = n - 1$ 个盘，原版hanoi算法满足不违反规则III
- 当 $k = n$ 时，原版算法分三步：
 1. `hanoi(n - 1,src, tmp, dst)` : 由归纳假设，该步骤不违反
 2. `n src→dst` : n 号盘移动到空柱B（因为第一步将较小的 $n - 1$ 个盘全部移到了tmp柱），一定不违反
 3. `hanoi(n-1, tmp,dst,src)` : 由归纳假设，该步骤不违反。

故原版算法不违法规则III

